



Databases and its Applications

Relational Modelling

Pooja T S
Computer Applications

Databases and its Applications

Relational Modelling

ER Model: Entities, Attributes, Keys, Relationships

Pooja T S

Computer Applications



Databases and its Applications

Introduction to Data Modelling – Overview

► Definition:

- Data Modelling is the process of analysing and defining data requirements needed to support business processes.
- It ensures that data objects and their relationships are represented accurately.

► Why Important?

- Converts unstructured business needs into structured form.
- Acts as a foundation for database design.

► Goal: Create a logical representation of data that supports consistency and scalability.



Databases and its Applications

Need for Data Modelling

► Key Reasons:

- Simplifies complex systems by visualising data components.
- Ensures data integrity by defining relationships and constraints early.
- Avoids redundancy and improves query performance in later stages.
- Provides a standard reference model for developers and analysts.

- **Without data modelling:** system may have duplicated, inconsistent or orphan data.



Databases and its Applications

Levels of Data Abstraction

► Three Major Levels:

- **Conceptual Level** — High-level description of data (entities, attributes, relationships).
Represented using the ER Model.
- **Logical Level** — Converts ER model into schema with tables, fields, and keys.
- **Physical Level** — Defines actual storage on disk (indexes, file structures, partitions).



- ▶ The ER model is the **first formal step** in the database design process.
 - Converts user requirements into a visual model.
 - Serves as blueprint for relational schema creation.
 - Provides a platform for feedback before implementation.
- ▶ **Key point:** ER diagrams are independent of any database software.



Databases and its Applications

Overview of the ER Model

- ▶ **Definition:** The **Entity-Relationship (ER) Model** is a high-level data model which represents the logical view of system from the data perspective, proposed by Peter Chen in 1976.
- ▶ **Purpose:**
 - Represent data in terms of entities, attributes and relationships.
 - Capture business rules visually before implementation.
- ▶ **Used For:**
 - System analysis, data requirement collection, and schema generation.



Databases and its Applications

Components of an ER Model

- ▶ The ER model consists of three main components:
 - **Entities** – Objects or concepts about which data is stored.
 - **Attributes** – Descriptive properties of entities or relationships.
 - **Relationships** – Logical associations between entities.
- ▶ **Output:** An ER Diagram (ERD) representing the conceptual schema.



Databases and its Applications

DIAGRAM ILLUSTRATION

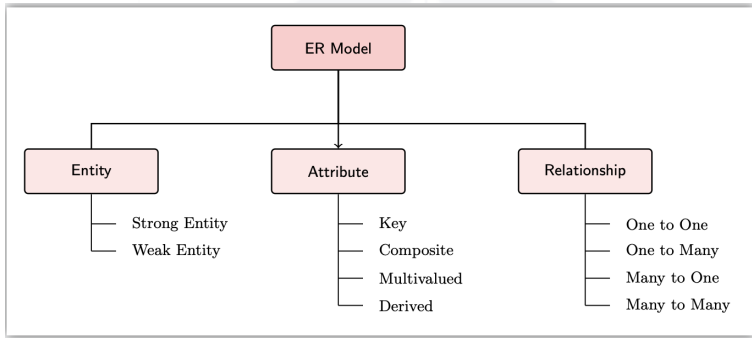


Figure: Components of ER Model



Databases and its Applications

ER Diagram Notations – Symbol Legend

Symbol	Diagram	Meaning / Description
Entity		Represents a real-world object or concept (e.g., <i>Student</i> , <i>Department</i>).
Weak Entity		An entity that depends on a strong entity for existence.
Relationship		Represents an association between entities.
Mandatory Relationship		Indicates that participation is required in the relationship.
Optional Relationship		Indicates optional participation in the relationship.



Databases and its Applications

ER Diagram Notations – Symbol Legend



Symbol	Diagram	Meaning / Description
Attribute		Describes a property or characteristic of an entity or relationship.
Key Attribute		Uniquely identifies an entity instance (underlined attribute).
Multivalued Attribute		Represents attributes with multiple values (e.g., <i>phone.no</i>).
Derived Attribute		An attribute computed from other attributes (e.g., <i>age</i> from <i>DOB</i>).
Partial Participation		Only some entities participate in the relationship.
Total Participation		All entities must participate in the relationship.



Databases and its Applications

Entities – Definition and Concept

- ▶ **Entity:** A distinguishable object (real or abstract) that exists and is represented in the database.
 - **Tangible:** e.g., Car, Student, Book.
 - **Intangible:** e.g., Course, Account, Department.
- ▶ Entities with similar attributes form an **Entity Set**.

Student



Databases and its Applications

Types of Entities – Strong vs Weak

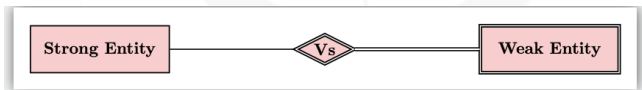
► Strong Entity:

- Has a primary key that uniquely identifies each instance.
- A strong entity is an entity type that has a key attribute.
- Example: Employee(EID, Name).

► Weak Entity:

- Cannot be uniquely identified by its attributes alone.
- Depends on a strong entity for identification.
- Example: Dependent(Name, Relation, EID).

► Notation: Weak entities → double rectangles.

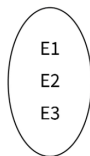




Databases and its Applications

Entity Sets and Instances

- ▶ **Entity Set:** Group of entities of the same type.
- ▶ **Entity Instance:** A single, unique object in the entity set.
- ▶ **Example:**
 - Entity Set – EMPLOYEE.
 - Instances – E1, E2, E3.



Entity Set



Databases and its Applications

Attributes – Definition and Role

- ▶ Attributes describe characteristics of entities or relationships.
 - Example: Student → (Name, id, phoneno, age).
 - Example: Book → (Title, Author, Price).
- ▶ Each attribute has a **Domain** – the set of valid values.



Databases and its Applications

Types of Attributes – I

► Simple Attribute:

- Attributes that cannot be further decomposed into sub-attributes are called simple attributes.
- Indivisible, atomic values.
- e.g., EmpID, RollNo.

► Composite Attribute:

- Can be subdivided into smaller parts.
- An attribute that is composed of many other attributes and can be decomposed into simple attributes
- e.g., Name = (FirstName, LastName).



Databases and its Applications

Types of Attributes – II

► **Single-valued Attribute:**

- Only one value per entity instance.
- e.g., DOB.

► **Multi-valued Attribute:**

- May have multiple values for one entity.
- e.g., PhoneNo.

► **Notation:** Double ellipse represents multi-valued attributes.



Databases and its Applications

Types of Attributes – III

► **Derived Attribute:**

- Computed from other attributes.
- e.g., $\text{Age} = \text{CurrentDate} \{ \text{DOB} \}$.

► **Null Attribute:**

- Represents missing or inapplicable data.
- Example: `PassportNo` for students without passports.

► **Notation:** Dashed ellipse indicates derived attributes.



Databases and its Applications

Attribute Domains and Notation

▶ Attribute Domain:

- Defines permissible values for an attribute.
- Example: Salary → Positive Integers.

▶ Notation Recap:

- Single ellipse – Simple Attribute.
- Double ellipse – Multivalued.
- Dashed ellipse – Derived.
- Underline – Key Attribute.



Databases and its Applications

Keys – Overview

► Purpose of Keys:

- Uniquely identify each entity instance.
- Prevent duplication and maintain data integrity.
- Serve as reference points for relationships.



Databases and its Applications

Types of Keys – I

- ▶ **Primary Key:** Uniquely identifies each entity.
- ▶ **Candidate Key:** All possible keys that can serve as unique identifiers.
- ▶ **Alternate Key:** Remaining candidate keys not chosen as primary.
- ▶ **Example:**
 - Entity: Student(StudentID, Email).
 - Primary Key – StudentID.
 - Alternate Key – Email.



Databases and its Applications

Types of Keys – II

► **Composite Key:**

- Formed by combining two or more attributes.
- Example: `CourseOffering(CourseID, Semester)`.

► **Foreign Key (Introduction):**

- Attribute(s) in one entity referring to the key of another.
- Used when mapping ER model to relational schema.

► **Note:** Proper key selection is crucial for preventing anomalies.



Databases and its Applications

Relationship – Concept

- ▶ **Relationship:** A logical connection between two or more entities.
 - Describes real-world interactions like:
 - Student enrolls in Course.
 - Faculty teaches Course.
- ▶ Represented by a diamond shape.



Databases and its Applications

Relationship Types – Introduction

► Definition:

- Relationship types define how entities are associated in terms of the number of instances that participate.
- These associations are also called **Mapping Cardinalities**.

► Four primary types:

- **One-to-One (1:1)**
- **One-to-Many (1:N)**
- **Many-to-One (N:1)**
- **Many-to-Many (M:N)**

► These mappings determine how tables or documents will later link together in database design.



Databases and its Applications

One-to-One (1:1) Relationship

► Definition:

- Each entity in set A is associated with at most one entity in set B, and vice versa.

► Examples:

- Each Student has one ID Card.
- Each Employee has one Parking Slot.

► Characteristics:

- Rare in practice — often merged into a single entity if mandatory on both sides.
- Useful when separating sensitive or optional data.



Databases and its Applications

One-to-Many (1:N) Relationship

► Definition:

- One entity in A can be related to many entities in B, but each entity in B is related to only one entity in A.

► Examples:

- One Department has many Employees.
- One Teacher teaches many Courses.

► Characteristics:

- The “one” side is often the parent entity.
- Most common relationship type in ER modelling.



Databases and its Applications

Many-to-One (N:1) Relationship

► Definition:

- Many entities in A relate to a single entity in B.
- Reverse perspective of a 1:N relationship.

► Examples:

- Many Students belong to one Department.
- Many Orders placed by one Customer.

► Use Case:

- Typically represented from the “many” side’s viewpoint (foreign key appears in the many entity).



Databases and its Applications

Many-to-Many (M:N) Relationship

► Definition:

- Entities in A can be associated with multiple entities in B, and vice versa.

► Examples:

- Students enroll in multiple Courses.
- Doctors treat multiple Patients.

► Characteristics:

- Cannot be directly implemented in relational databases.
- Converted into an **Associative Entity** (bridge table) with two 1:N relationships.



Databases and its Applications

Relationship Notations

Symbol	Diagram	Meaning / Description
One to One	—————	Exactly one instance on each side.
One to Many (Mandatory)	— —————>	One on the left relates to one or many on the right; participation on the left is mandatory.
Many	<—————	Many on the left (crow's foot), unconstrained on the right.
One or More (Mandatory)	<— —————	At least one (mandatory) and possibly many on the left.
One and Only One (Mandatory)	— —————	Exactly one on the left; participation is mandatory.
Zero or One (Optional)	—○—————	Optional participation: zero or one on the left.
Zero or Many (Optional)	<—○—————	Optional participation with many allowed on the left.



- ▶ Illustrating all mapping types in one scenario:
 - 1:1 – Each Student has one LibraryCard.
 - 1:N – One Department offers many Courses.
 - N:1 – Many Courses taught by one Professor.
 - M:N – Students register for multiple Courses.



► Summary:

- Mapping types describe the quantitative association between entities.
- Four mappings: 1:1, 1:N, N:1, and M:N.
- These guide how foreign keys or embedded references are designed in implementation.



Databases and its Applications

Relationships – Types Overview

► What is a relationship type?

- A template that specifies how entity sets are associated.
- Described by **degree**, **roles**, and optional **relationship attributes**.

► We cover:

- **Unary (recursive)**, **Binary**, **Ternary**, **N-ary** relationships.
- **Identifying vs Non-identifying** (strong vs weak) relationships.
- **Associative entities** and **role names**.



Databases and its Applications

Relationship Degree – Definition

- ▶ **Degree:** Number of distinct entity sets that participate.
 - **Unary (1)** – one entity set participates.
 - **Binary (2)** – two entity sets (most common).
 - **Ternary (3)** – three entity sets.
 - **N-ary (n)** – $n \geq 4$ entity sets.
- ▶ **Note:** Degree is different from *cardinality/participation* (covered next sessions).



Databases and its Applications

Unary (Recursive) Relationships

- ▶ **Definition:** An entity set relates to itself.
- ▶ **Examples:**
 - Employee supervises Employee.
 - Student mentors Student.
 - Part assembledFrom Part (BOM).
- ▶ **Role Names:** Use [Supervisor/Subordinate](#), [Mentor/Mentee](#), etc., to avoid ambiguity.



Databases and its Applications

Unary – Design Tips

- ▶ When to use:
 - Hierarchies or networks within one entity set.
 - Peer-to-peer links among the same entity type.
- ▶ Modelling guidance:
 - Add **role names** & **relationship attributes** (e.g., SinceDate).
 - Keep the relationship separate; do not duplicate attributes inside the entity.



Databases and its Applications

Binary Relationships

- ▶ **Definition:** Association between **two** entity sets.
- ▶ Typical examples:
 - Customer places Order
 - Teacher teaches Course
 - Book publishedBy Publisher
- ▶ **Attributes on relationship:**
 - Enrollment(Student, Course, Grade)
 - Works_On(Employee, Project, Hours)



Databases and its Applications

Binary – Modelling Guidance

- ▶ Use a relationship when:
 - The association is **conceptually separate** from either entity.
 - The link has its own **attributes** (e.g., Grade, Hours, StartDate).
- ▶ **Avoid:** Storing relationship facts as repeated attributes in an entity (leads to redundancy).



Databases and its Applications

Ternary Relationships

- ▶ **Definition:** Association among **three** entity sets.
- ▶ Classic example:
 - Supplier supplies Part to Project with attribute Qty.
- ▶ Why needed:
 - The business rule involves **all three** simultaneously (not pairwise only).



Ternary – When Binary Decomposition Fails

- ▶ **Caution:** Replacing a true ternary with multiple binaries can lose semantics.
- ▶ **Example:**
 - Supplier{Part, Supplier{Project, Part{Project triples do **not** enforce “this supplier supplied this part to *this project* with Qty”.
- ▶ **Guideline:** Use a **ternary** when the attribute/value (e.g., Qty) is defined only for the **combination of three**.



Databases and its Applications

N-ary Relationships

- ▶ **Definition:** Association among four or more entity sets.
- ▶ Use cases:
 - Expert evaluates Product on Criterion for Market with Score.
- ▶ Modelling note:
 - Consider whether the relationship is better as an **associative entity** with links to the n entities.



Identifying vs Non-identifying Relationships

► Identifying (Strong) Relationship:

- Used to identify a **weak entity**.
- Notation: **double diamond**.
- Example: Employee $\leftrightarrow$ Dependent (identified by (EID, DepName)).

► Non-identifying (Regular) Relationship:

- Association that does **not** determine the identity of either entity.
- Notation: **single diamond**.



- ▶ **Option A:** Keep a **relationship with attributes** (e.g., Enrollment has Grade).
- ▶ **Option B:** Reify the relationship into an **associative entity**:
 - Useful when the association itself has **identity**, multiple attributes, or other relationships.
- ▶ **Tip:** Prefer associative entity when you will attach **more relationships** to the link itself (e.g., OrderItem).



Databases and its Applications

Choosing the Right Relationship Type

► Decision cues:

- Is the link among **one**, **two**, or **three+** entities? → choose degree.
- Does the link have its **own attributes**? → keep as relationship or associative entity.
- Does the link **identify** a weak entity? → identifying (double diamond).

- **Preview:** Cardinality and participation constraints will refine these designs next.



Databases and its Applications

Roles and Role Names

- ▶ **Roles:** The *function* an entity plays in a relationship.
- ▶ Needed when:
 - Same entity set participates more than once (**recursive**).
 - Symmetric/asymmetric semantics must be clarified.
- ▶ Examples:
 - Employee (Supervisor) and Employee (Subordinate).
 - Person (Parent) and Person (Child).



- ▶ Attach to the **relationship** when the fact:
 - **Describes the association** (not either entity alone).
 - Varies per pair/tuple of participating entities (e.g., Hours per Employee{Project}).
- ▶ **Do not:** Copy the same attribute into both entities (redundancy, inconsistency).



► Watch out for:

- Replacing a true **ternary** with binary links (loses meaning).
- Omitting **role names** in recursive relationships.
- Turning a relationship into attributes inside entities (denormalised design).
- Using identifying relationships when the entity is actually strong.



Thank You

Pooja T S

Assistant Professor

Department of Computer Applications

poojats@pes.edu

080-26721983 Extn: 233